

ENTERPRISE FULLSTACK ARCHITECTURE HANDBOOK

BookFlow Studio

Comprehensive Technical Architecture, Component Specifications, Database Schema, and Execution Guide for the Fullstack Student Library & Community Platform.

BACKEND ARCHITECTURE

ASP.NET Core 10 Web API

EF Core 10, SQL Server, JWT

FRONTEND PLATFORM

React 19 + Vite

Web Speech, PDF Engine, Router 7

AUTHOR & ENVIRONMENT

Production Grade

Role-Based (Admin & Student)

1. Executive Summary & System Overview

BookFlow Studio is a modern, enterprise-grade full-stack reading and library management ecosystem. The platform seamlessly bridges catalog administrative operations (books, multi-currency pricing, multi-language mappings, cover palettes) with an engaging social reading laboratory for students (interactive speech voice narration, PDF streaming, focus reading timers, digital badges, and live friend activity feeds).

Core Technical Mission

Deliver high-performance data processing with clean separation of concerns, robust security (BCrypt + JWT + Role-based Authorizations), seamless API proxies, and interactive modern UI components.

System Architecture Diagram

Client Layer (Frontend - React 19 + Vite)

SPA Architecture, React Router DOM v7, Context State, Web Speech Synthesis API, PDF Blob Streamer

Port 5173

↕ **Axios Reverse Proxy (/api route) & Bearer JWT Header**

API Gateway & Application Layer (ASP.NET Core 10 Web API)

Controllers, FluentValidation, AutoMapper Profiles, Request Logging Middleware, Serilog File Logger

Port 5142 / 7103

↕ **Entity Framework Core 10 (Code-First / DbContext)**

Relational Database Layer (Microsoft SQL Server)

BookDb: Normalized Relational Tables, Composite Junctions, Identity Seeds, Audit Logs, Query Filters

Server: ERAIT1104

2. Complete Technology Stack Matrix

DOMAIN	TECHNOLOGY / LIBRARY	VERSION	PRIMARY RESPONSIBILITY
Backend Runtime	.NET 10.0 (C# 14)	10.0.401	High-performance server runtime & ASP.NET Core Web API
ORM & Database	EF Core + MS SQL Server	10.0.0	Relational schema mapping, migrations, LINQ queries & audit tracking
Authentication	JWT Bearer + BCrypt.Net	4.0.3	Token generation, role verification, password hashing & logout
Object Mapping	AutoMapper	12.0.1	Decoupled entity-to-DTO and DTO-to-entity projections
Validation	FluentValidation.AspNetCore	11.3.0	Strict declarative business rule validation for incoming requests
Diagnostics	Serilog + Console + File Sink	4.0.0	Structured daily rolling log files with request execution time
Frontend Framework	React.js (Hooks + SPA)	19.0.0	Component-driven UI, state management & lifecycle hooks
Build Tool	Vite	8.3.0	Blazing-fast HMR dev server & Rollup production bundle compiler
Audio Engine	Web Speech API (Synthesis)	Native	Multi-voice, pitch, volume & speed text-to-speech audio narration
Document Engine	HTML5 Canvas & Blob Streamer	Native	PDF thumbnail rendering, chunked binary download & reader modal

3. Backend Architecture & Database Layer

The backend is structured under clean architecture principles with distinct boundaries between Controllers, Repositories, Domain Models, DTOs, and Services.

3.1 Database Schema & Entities

TABLE NAME	PRIMARY KEY	FOREIGN KEYS / RELATIONSHIPS	KEY PURPOSE
Books	Id (int)	Junction to Languages, Colors, Prices	Core book catalog (Title, Description, Pages, PdfFilePath, IsActive)
Currency	id (int)	1-to-Many with BookPrices	Currency codes (INR, USD, EUR, Dinar) & exchange symbol mappings
BookPrices	Id (int)	BookId → Books, CurrencyId → Currency	Multiple price assignments per book in distinct currencies
Languages	Id (int)	Junction with BookLanguages	Available languages (Hindi, English, Tamil, Urdu, etc.)
BookLanguages	BookId + LanguageId	Composite PK	Many-to-Many junction enabling multi-lingual book editions
Colors	Id (int)	Junction with BookColors	Theme color palette records (Hex, Names) for book covers
BookColors	BookId + ColorId	Composite PK	Many-to-Many junction mapping books to color aesthetics
Users	Id (int)	Friendships, Activities	Accounts (Name, Email, passwordHash, Role='Admin' 'User', Lockout)
Friendships	Id (int)	UserId, FriendId → Users	Social network graph (Status: 0=Pending, 1=Accepted)
UserActivities	Id (int)	UserId → Users, BookId → Books	Live feed timeline (RatedBook, FinishedReading, AddedNote, etc.)
AuditLogs	Id (int)	Autonomous Identity	Temporal audit trail storing JSON diffs of INSERT/UPDATE/DELETE

3.2 REST API Controller Endpoints

- **AuthController:** POST /api/Auth/Register (New account creation), POST /api/Auth/login (Credentials verification, token generation, 5-attempt logout security).
- **BookController:** GET /api/Book (Eager-loaded book catalog), GET /api/Book/{id} (Single book specs), POST /api/Book (Admin book creation), PUT /api/Book/{id} (Admin update), DELETE /api/Book/{id} (Admin soft delete), POST /api/Book/{id}/upload-pdf, GET /api/Book/{id}/read (Blob binary reader).

- **CurrencyController:** GET /api/currencies, POST /api/currencies, PUT /api/currencies/{id}, DELETE /api/currencies/{id}.
- **LanguageController:** GET /api/Language, POST /api/Language, PUT /api/Language/{id}, DELETE /api/Language/{id}.
- **ColorController:** GET /api/colors, POST /api/colors, PUT /api/colors/{id}.
- **FeedController:** GET /api/Feed/{userId} (Live aggregated stream of user + accepted friends), POST /api/Feed/activity (Log reading milestone).
- **FriendController:** POST /api/Friend/request, POST /api/Friend/accept, POST /api/Friend/reject, GET /api/Friend/list/{userId}, GET /api/Friend/pending/{userId}.

4. Frontend Architecture & User Experience

Built with React 19, the frontend delivers a high-speed Single Page Application (SPA) with zero page reloads, fluid CSS keyframe transitions, and modular UI structure.

4.1 Application Component Breakdown

COMPONENT	FILE PATH	KEY FEATURES & USER CAPABILITIES
Navbar	src/Components/Navbar.jsx	Desktop/Mobile responsive menu, Spotlight quick search (Ctrl+K), Live red pulsing friend request badge counter.
Dashboard	src/Components/Dashboard/Dashboard.jsx	Executive metric cards (Total Books, Currencies, Languages, Colors), interactive quick shortcuts, active catalog feed.
Book Catalog	src/Components/Book/BookList.jsx	Grid cards with dynamic gradients, 1.5s debounced multi-field search, filter tabs (All, PDF, Priced, Multi-lang), pagination.
Book Details	src/Components/Book/BookDetails.jsx	Rich technical sheet, INR/USD/EUR multi-currency badges, PDF reader modal, PDF cover canvas generator, trigger studios.
Voice Narration	src/Components/Book/BookAudioStudio.jsx	Web Speech API narration studio: Play/Pause/Resume, custom voice picker, speed multiplier, pitch tuning, waveform visualizer.
Reading Lab	src/Components/Book/BookReadingLab.jsx	Distraction-free reading environment, live stopwatch timer, reading session notes saver, custom font scaling & theme toggles.
3D Digital Card	src/Components/Book/BookDigitalCardModal.jsx	Interactive holographic book card generator with QR verification specs, cover glow effects, and downloadable assets.
Social Feed & Friends	src/Components/Social/SocialFeed.jsx	Live friend activity feed, send friend invitation modal, review incoming requests, and top-screen celebratory notifications .
Auth Provider	src/context/AuthContext.jsx	JWT token state, automatic role claim extraction, multi-account switcher storage, 2-stage animated login progress loader.

4.2 Voice Narration & Speech Studio Implementation

The voice narration module (BookAudioStudio.jsx) utilizes the browser's native **Web Speech Synthesis API**. It intelligently tokenizes book descriptions, extracts synthesized voices matching user locale, and dynamically manages speech playback states.

```
// Sample voice synthesis logic from BookAudioStudio.jsx
const synth = window.speechSynthesis;
const utterance = new SpeechSynthesisUtterance(book.description);
utterance.rate = playbackSpeed; // e.g., 0.75x, 1.0x, 1.25x, 1.5x
```

```
utterance.pitch = voicePitch; // 0.5 to 1.5 if (selectedVoice) utterance.voice = selectedVoice;  
synth.speak(utterance);
```

4.3 Real-Time Notification & Friend Request System

The social network system provides instantaneous visual feedback across two distinct touchpoints:

- **Navbar Badge Indicator:** `Navbar.jsx` polls `/api/Friend/pending/{userId}` on an interval and listens to global `friend-requests-updated` events. Whenever a friend request is received, a glowing red notification count badge appears on the **Feed & Friends** menu item.
- **Top Floating Action Popups:** When a user sends or accepts a request in `SocialFeed.jsx`, an elegant obsidian-glass pop-up slides down from the top of the viewport with neon indicator lines, user avatars, and status badges (REQUEST SENT / CONNECTED).

5. Security, Authentication & Deployment Guide

5.1 Authentication & Security Pipeline

- 1. **Password Security:** User passwords are never stored in plaintext. They are salted and hashed using **BCrypt.Net-Next** (Work Factor 11).
- 2. **Account Protection & Lockout:** If 5 consecutive invalid login attempts occur, the account is automatically locked for 15 minutes (`LockoutEnd =.UtcNow.AddMinutes(15)`) and returns HTTP 423 Locked.
- 3. **Stateless JWT Tokens:** Successful authentication issues a HMAC-SHA256 signed JSON Web Token containing claims for Sub (`UserId`), Email, and Role (`"Admin" | "User"`) with a 7-day UTC validity.
- 4. **Role-Based Route Guards:** The frontend utilizes `<ProtectedRoute>` for authenticated members and `<AdminRoute>` for administrator-only operations (Create, Edit, Delete).

5.2 Quick Run & Execution Commands

SERVICE	WORKING DIRECTORY	COMMAND	LISTENING PORT
Backend API	Desktop/EntityFrameworkCore/.../DbOperationsWithEfcoreApp	dotnet run	http://localhost:5142
Frontend SPA	c:\Users\Administartor\book-student-frontend	npm run dev	http://localhost:5173
Frontend Build	c:\Users\Administartor\book-student-frontend	npm run build	Generates /dist

✔ Platform Verification Summary

The entire application is fully configured, builds cleanly with 0 errors, and provides seamless end-to-end integration across SQL Server, ASP.NET Core 10, and React 19.